

如，即使类 BC 例程(2 个加载和 1 个存储)在内循环中比类 AB 例程(2 个加载)执行更多的内存引用，类 BC 例程(每次迭代有 0.5 个不命中)比类 AB 例程(每次迭代有 1.25 个不命中)性能还是要好很多。

- 对于大的 n 值，最快的一对版本(kij 和 ikj)的性能保持不变。虽然这个数组远大于任何 SRAM 高速缓存存储器，但预取硬件足够聪明，能够认出步长为 1 的访问模式，而且速度足够快能够跟上内循环中的内存访问。这是设计这个内存系统的 Intel 的工程师所做的一项极好成就，向程序员提供了甚至更多的鼓励，鼓励他们开发出具有良好空间局部性的程序。

网络旁注 MEM:BLOCKING 使用分块来提高时间局部性

有一项很有趣的技术，称为分块(blocking)，它可以提高内循环的时间局部性。分块的大致思想是将一个程序中的数据组织成的大片(chunk)，称为块(block)。(在这个上下文中，“块”指的是一个应用级的数据组块，而不是高速缓存块。)这样构造程序，使得能够将一个片加载到 L1 高速缓存中，并在这个片中进行所需的所有的读和写，然后丢掉这个片，加载下一个片，依此类推。

与为提高空间局部性所做的简单循环变换不同，分块使得代码更难阅读和理解。由于这个原因，它最适合于优化编译器或者频繁执行的库函数。由于 Core i7 有完善的预取硬件，分块不会提高矩阵乘在 Core i7 上的性能。不过，学习和理解这项技术还是很有趣的，因为它是一个通用的概念，可以在一些没有预取的系统上获得极大的性能收益。

6.6.3 在程序中利用局部性

正如我们看到的，存储系统被组织成一个存储设备的层次结构，较小、较快的设备靠近顶部，较大、较慢的设备靠近底部。由于采用了这种层次结构，程序访问存储位置的实际速率不是一个数字能描述的。相反，它是一个变化很大的程序局部性的函数(我们称之为存储器山)，变化可以有几个数量级。有良好局部性的程序从快速的高速缓存存储器中访问它的大部分数据。局部性差的程序从相对慢速的 DRAM 主存中访问它的大部分数据。

理解存储器层次结构本质的程序员能够利用这些知识编写出更有效的程序，无论具体的存储系统结构是怎样的。特别地，我们推荐下列技术：

- 将你的注意力集中在内循环上，大部分计算和内存访问都发生在这里。
- 通过按照数据对象存储在内存中的顺序、以步长为 1 的来读数据，从而使得你程序中的空间局部性最大。
- 一旦从存储器中读入了一个数据对象，就尽可能多地使用它，从而使得程序中的时间局部性最大。

6.7 小结

基本存储技术包括随机存储器(RAM)、非易失性存储器(ROM)和磁盘。RAM 有两种基本类型。静态 RAM(SRAM)快一些，但是也贵一些，它既可以用做 CPU 芯片上的高速缓存，也可以用作芯片下的高速缓存。动态 RAM(DRAM)慢一点，也便宜一些，用做主存和图形帧缓冲区。即使是在关电的时候，ROM 也能保持它们的信息，可以用来存储固件。旋转磁盘是机械的非易失性存储设备，以每个位很低的成本保存大量的数据，但是其访问时间比 DRAM 长得多。固态硬盘(SSD)基于非易失性的闪存，对某些应用来说，越来越成为旋转磁盘的具有吸引力的替代产品。

一般而言，较快的存储技术每个位会更贵，而且容量更小。这些技术的价格和性能属性正在以显著

不同的速度变化着。特别地, DRAM 和磁盘访问时间远远大于 CPU 周期时间。系统通过将存储器组织成存储设备的层次结构来弥补这些差异, 在这个层次结构中, 较小、较快的设备在顶部, 较大、较慢的设备在底部。因为编写良好的程序有好的局部性, 大多数数据都可以从较高层得到服务, 结果就是存储系统能以较高层的速度运行, 但却有较低层的成本和容量。

程序员可以通过编写有良好空间和时间局部性的程序来显著地改进程序的运行时间。利用基于 SRAM 的高速缓存存储器特别重要。主要从高速缓存取数据的程序能比主要从内存取数据的程序运行得快得多。

参考文献说明

内存和磁盘技术变化得很快。根据我们的经验, 最好的技术信息来源是制造商维护的 Web 页面。像 Micron、Toshiba 和 Samsung 这样的公司, 提供了丰富的当前有关内存设备的技术信息。Seagate 和 Western Digital 的页面也提供了类似的有关磁盘的有用信息。

关于电路和逻辑设计的教科书提供了关于内存技术的详细信息[58, 89]。IEEE Spectrum 出版了一系列有关 DRAM 的综述文章[55]。计算机体系结构国际会议(ISCA)和高性能计算机体系结构(HPCA)是关于 DRAM 存储性能特性的公共论坛[28, 29, 18]。

Wilkes 写了第一篇关于高速缓存存储器的论文[117]。Smith 写了一篇经典的综述[104]。Przybylski 编写了一本关于高速缓存设计的权威著作[86]。Hennessy 和 Patterson 提供了对高速缓存设计问题的全面讨论[46]。Levinthal 写了一篇有关 Intel Core i7 的全面性能指南[70]。

Stricker 在[112]中介绍了存储器山的思想, 作为对存储器系统的全面描述, 并且在后来的工作描述中非正式地提出了术语“存储器山”。编译器研究者通过自动执行我们在 6.6 节中讨论过的那些手工代码转换来增加局部性[22, 32, 66, 72, 79, 87, 119]。Carter 和他的同事们提出了一个高速缓存可知晓的内存控制器(cache-aware memory controller)[17]。其他的研究者开发出了高速缓存不知晓的(cache oblivious)算法, 它被设计用来在不明确知道底层高速缓存存储器结构的情况下也能运行得很好[30, 38, 39, 9]。

关于构造和使用磁盘存储设备也有大量的论著。许多存储技术研究者找寻方法, 将单个的磁盘集成成更大、更健壮和更安全的存储池[20, 40, 41, 83, 121]。其他研究者找寻利用高速缓存和局部性来改进磁盘访问性能的方法[12, 21]。像 Exokernel 这样的系统提供了更多的对磁盘和存储器资源的用户级控制[57]。像安德鲁文件系统[78]和 Coda[94]这样的系统, 将存储器层次结构扩展到了计算机网络和移动笔记本电脑。Schindler 和 Ganger 开发了一个有趣的工具, 它能自动描述 SCSI 磁盘驱动器的构造和性能[95]。研究者正在研究构造和使用基于闪存的 SSD 的技术[8, 81]。

家庭作业

- 6.22 假设要求你设计一个每条磁道位数固定的旋转磁盘。你知道每条磁道的位数是由最里层磁道的周长决定的, 可以假设它就是中间那个圆洞的周长。因此, 如果你把磁盘中间的洞做得大一点, 每条磁道的位数就会增大, 但是总的磁道数会减少。如果用 r 来表示盘面的半径, $x \cdot r$ 表示圆洞的半径, 那么 x 取什么值能使这个磁盘的容量最大?
- 6.23 估计访问下面这个磁盘上扇区的平均时间(以 ms 为单位):

参数	值
旋转速率	15 000RPM
$T_{\text{avg seek}}$	4 ms
平均扇区数/磁道	800

- 6.24 假设一个 2MB 的文件, 由 512 个字节的逻辑块组成, 存储在具有下述特性的磁盘驱动器上: